

Data Analysis

Data Imbalance and Text Encodings

CentraleDigitalLab@Nice

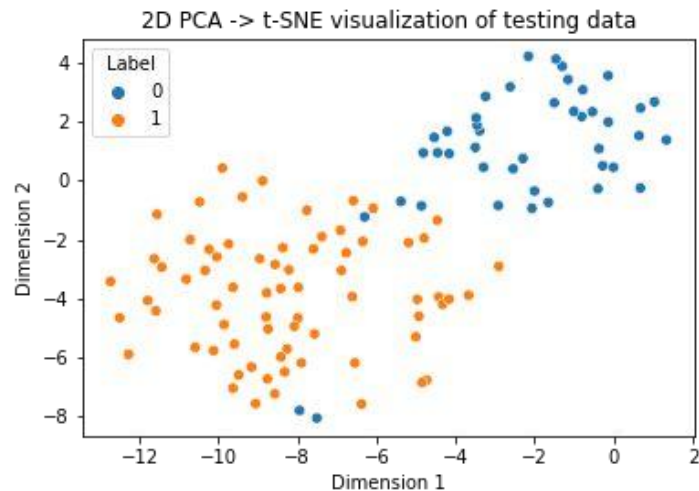
Pipeline + PCA + T-SNE

It is a common practice of using PCA followed by **t-SNE for error analysis** to check how good our features are.

Visualizing the test data in this way helps to **check the separability of the classes** in the transformed space.

Misclassified observations could inform further steps in the analysis or model tuning.

Decision Boundary Plot with TSNE



Demo with notebook
07__pca_with_tsne.ipynb

Imbalanced datasets

Data Imbalance refers to an **unequal distribution of classes** within a dataset.

In a binary classification problem, it's where **one class significantly outnumbers the other**. (Normally the positive class).

Example:

In fraud detection, fraudulent transactions (positive class) are far less common than non-fraudulent ones (negative class).

Imbalanced datasets

Consequences of Data Imbalance:

- **Model Bias:** Models trained on imbalanced data tend to be biased towards the majority class. They might **simply predict the majority class for all inputs** to minimize the error.
- **Poor Generalization:** The model might perform poorly on unseen data.
- **Misleading Accuracy:** High accuracy score might be misleading as it could be merely reflecting the underlying class distribution.
- **Reduced Sensitivity:** The model may have reduced sensitivity (true positive rate) for the minority class.

Data Augmentation

Generation of **new samples from the existing data** to enhance the size and variability of the dataset.

Methods:

- For image data: rotation, flipping, scaling, cropping, and color variations.
- For text data: synonym replacement, back-translation, and sentence shuffling.

Data Augmentation

Generation of **new samples from the existing data** to enhance the size and variability of the dataset.

Benefits:

- Enhances the model's ability to generalize.
- Helpful in scenarios where **collecting more data is not feasible**.

Oversampling

Increasing the number of minority class instances to balance the class distribution.

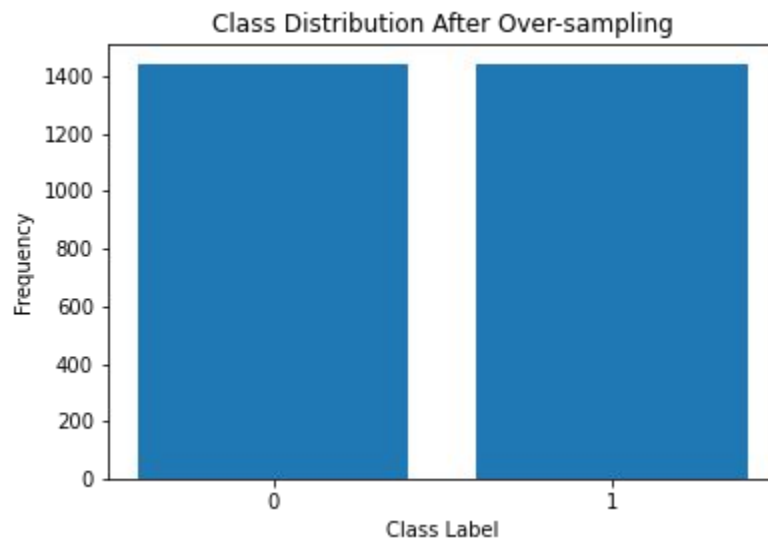
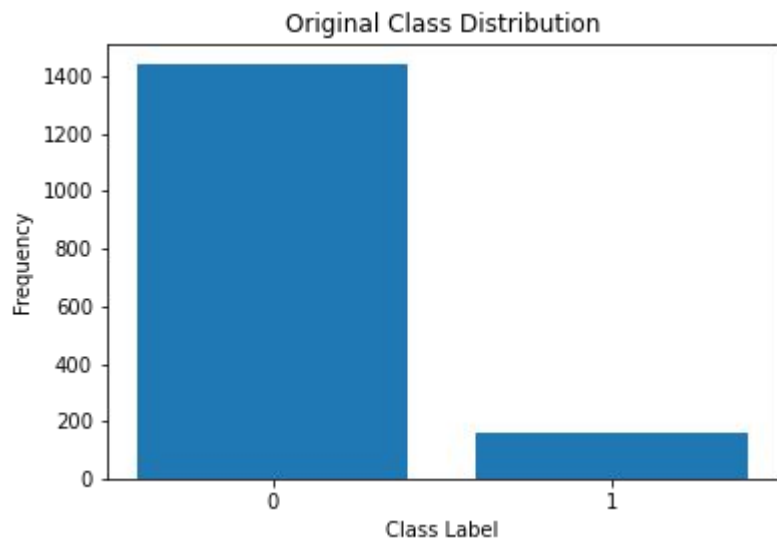
Algorithm: Random Over-sampler (ROS): **Randomly duplicates instances** of the minority class or generates synthetic examples.

Benefits: No loss of data.

Drawbacks: **May lead to overfitting** if synthetic examples do not sufficiently represent the true data distribution.

Oversampling

Increasing the number of minority class instances to balance the class distribution.



Undersampling

Reducing the number of majority class instances to balance the class distribution.

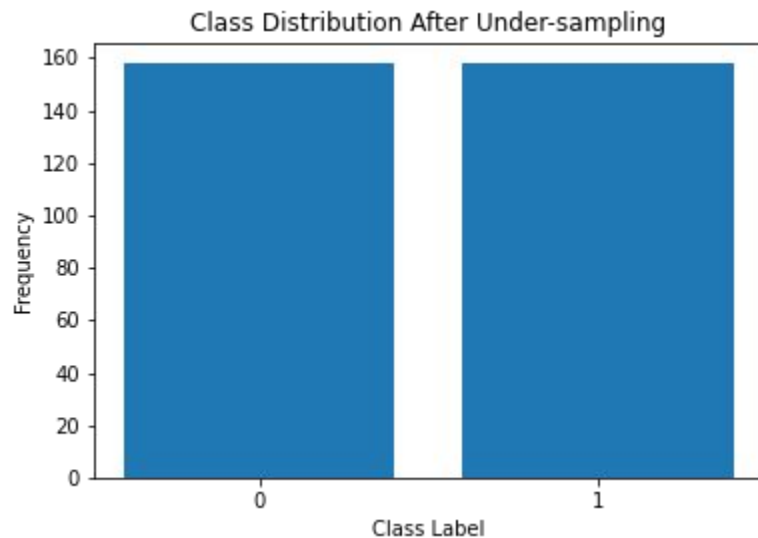
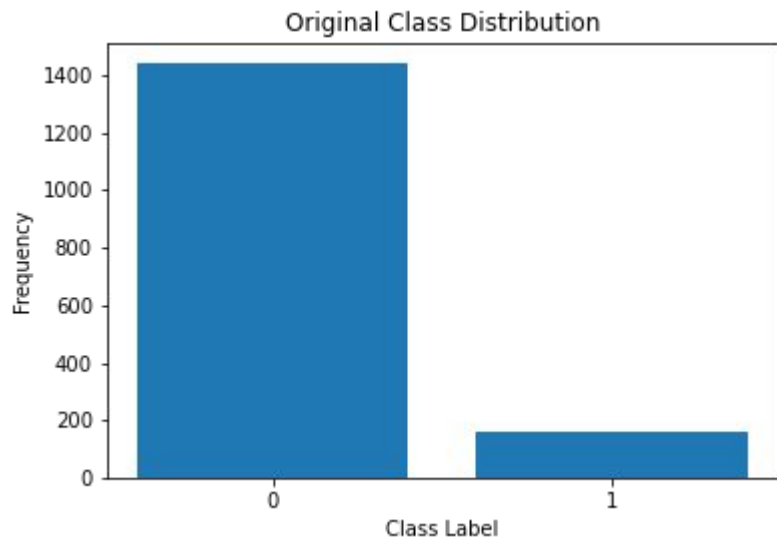
Algorithm: Random Under-sampler (RUS): **Randomly removes instances** of the majority class until a desired balance is achieved.

Benefits: Quick and easy to implement.

Drawbacks: **Loss of potentially important data** from the majority class.

Undersampling

Reducing the number of majority class instances to balance the class distribution.

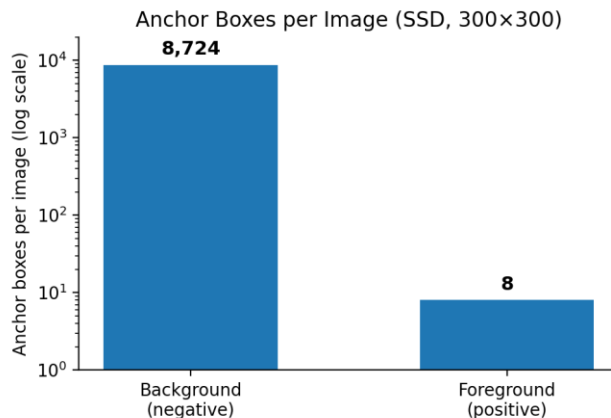


Class Imbalance in Object Detection

Object detectors must evaluate **thousands of candidate regions per image** — most contain no object.

SSD generates **8,732 anchor boxes per image**, but only a handful are true objects.

The foreground:background ratio can exceed **1:1000** — far more extreme than typical classification imbalance.



Hard Negative Mining

Hard negative mining chooses which **negative (background) examples** a detector trains on, instead of using every negative or a random subset.

Only the **hardest negatives** — background regions the model is currently most confident (and wrong) about — are kept for training.

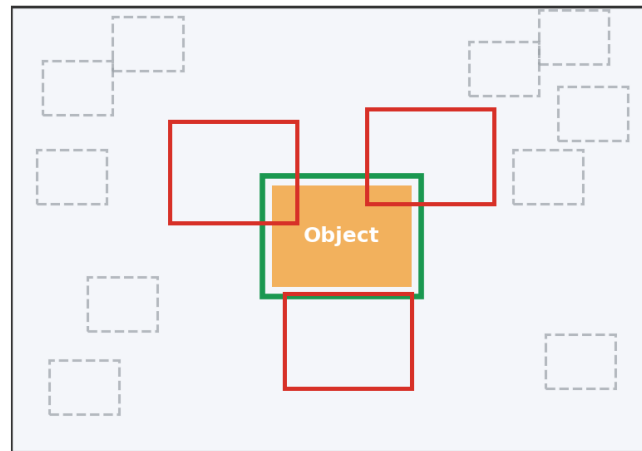
Why “hard”? These negatives produce high loss — they resemble the object or sit near its boundary — unlike **“easy” negatives**, already correctly rejected, which contribute little useful gradient.

Hard Negative Mining — How It Works

Steps used by hard negative mining:

- **Compute loss** for every negative anchor.
- **Rank** negatives by loss, highest first.
- **Select** top-k hardest (e.g. 3:1 ratio to positives).
- **Train** on hard negatives + all positives.

One Image: Positive, Hard, and Easy Anchor Boxes



- Positive anchor (object)
- Hard negative (kept)
- Easy negative (discarded)

Handling Imbalance via BCE Loss

Binary Cross-Entropy (BCE) loss compares a predicted probability p against the true label y :

$$\text{BCE}(p, y) = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

With extreme class imbalance, the loss is **dominated by the many easy background terms**, even though each one is individually small.

Weighted BCE introduces a class weight α to rebalance each class's contribution:

$$\text{Weighted BCE} = -[\alpha \cdot y \cdot \log(p) + (1-\alpha) \cdot (1-y) \cdot \log(1-p)]$$

α is set **inversely proportional to class frequency** (higher weight on the rare foreground class), so minority-class errors contribute proportionally more to the total loss — complementing hard negative mining.

Demo with notebook

08__imbalanced_datasets.ipynb

Text Encoding

The process of **converting text data into numerical data** that can be used by machine learning algorithms.

Machine learning models understand numbers, not text. Text encoding bridges this gap, **enabling models to analyze and learn from text data**.

Encoding extracts features from text, capturing the information necessary for the model to make predictions or understand patterns.

Bag of Words

Bag of Words (BoW) is a text encoding technique that represents text data as a **"bag" of individual words**.

- Tokenization: Split text into words (tokens).
- Vocabulary Building: Create a vocabulary of unique words.
- Encoding: **Count the occurrence of each word** in the text and represent the text as a vector of these counts.

Bag of Words

Bag of Words (BoW) is a text encoding technique that represents text data as a **"bag" of individual words**.

Advantages:

- Simple and easy to understand.
- Effective for text classification tasks.

Drawbacks:

- Ignores the order of words and context.
- Can lead to a high-dimensional sparse matrix.

Bag of Words — Example

Consider two example documents:

Doc1: "I love programming."

Doc2: "Programming is fun."

Step 1 — Vocabulary: list every **unique word** across both documents.

Vocabulary = [I, love, programming, is, fun]

Step 2 — Vectorization: count how many times each vocabulary word appears in each document.

Bag of Words — Example

Counting each vocabulary word's occurrences in each document gives:

Document	I	love	programming	is	fun
Doc1	1	1	1	0	0
Doc2	0	0	1	1	1

Even though "programming" appears in both documents, BoW gives it **the same weight as any other word** — it only looks at raw frequency.

TF-IDF

TF-IDF is an encoding technique that **weighs the importance of words** in a document relative to a collection of documents.

- **Term Frequency (TF)**: Count the occurrence of each word in a document.
- **Inverse Document Frequency (IDF)**: Logarithmically scale the inverse fraction of the documents that contain the word.
- **Encoding**: Multiply TF by IDF to obtain the TF-IDF weight for each word in a document.

TF-IDF

TF-IDF is an encoding technique that **weighs the importance of words** in a document relative to a collection of documents.

Advantages:

- Reduces the weight of common words.
- Highlights important words unique to a document.

Drawbacks:

- More complex than BoW.
- Still disregards the order of words.

TF-IDF — Example

TF-IDF first computes each word's **document frequency (DF)** — how many of the $N = 2$ documents contain it:

Word	DF (docs containing it)	IDF = $\log(2 / \text{DF})$
I	1	0.693
love	1	0.693
programming	2	0.000
is	1	0.693
fun	1	0.693

A word appearing in every document gets **IDF = 0** — it carries no distinguishing power.

TF-IDF — Example

Multiplying each word's TF by its IDF gives the final TF-IDF vectors:

Document	I	love	programming	is	fun
Doc1	0.693	0.693	0	0	0
Doc2	0	0	0	0.693	0.693

Notice "programming" **drops to 0** in both documents — despite appearing twice overall, TF-IDF correctly recognizes it carries no distinguishing information, while "love", "is", and "fun" are boosted.

Word Embeddings: Word2Vec & GloVe

Word embeddings represent each word as a dense vector (e.g. 100–300 dimensions), learned so that words with **similar meaning end up close together** in that space.

- **Word2Vec:** learns embeddings by predicting a word from its surrounding context, using a shallow neural network.
- **GloVe:** learns embeddings from global word co-occurrence statistics across the whole corpus.

Like BoW and TF-IDF, **a single embedding still has no notion of where a word sits in a sentence** — order isn't captured yet.

N-grams: Capturing Local Word Order

An **n-gram** is a contiguous sequence of n words, used as a single token instead of one word at a time.

- **Bigram ($n=2$):** "I love programming" $\rightarrow$ ["I love", "love programming"]
- **Trigram ($n=3$):** "I love programming" $\rightarrow$ ["I love programming"]

Underneath, an n-gram model is **still just Bag of Words or TF-IDF** — only what counts as a “word” changes, from single words to n-word sequences.

The Markov Assumption

The Markov assumption states that the probability of the next item **depends only on a fixed, limited window of recent history** — not the entire sequence before it.

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-k}, \dots, w_{i-1})$$

- **First-order (k=1):** $P(w_i \mid w_{i-1})$ — one word of context (bigram).
- **Second-order (k=2):** $P(w_i \mid w_{i-2}, w_{i-1})$ — two words of context (trigram).

A **simplifying approximation** — fewer parameters, less training data needed.

The N-gram Probability Model

N-grams are also the basis of **n-gram language models**, which estimate a sentence's probability using the **chain rule of probability**:

$$P(w_1, \dots, w_m) = \prod_i P(w_i \mid w_1, \dots, w_{i-1})$$

Since conditioning on the full history is impractical, the **n-gram (Markov) assumption** limits context to the previous $n-1$ words:

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-n+1}, \dots, w_{i-1})$$

Bigram model (n=2): $P(w_i \mid w_{i-1})$ — each word depends only on the one before it.

Sequence-Aware Models

Beyond n-grams, some architectures process word order as **part of how they compute**, not just what they count:

- **RNNs / LSTMs:** process tokens one at a time, carrying a hidden state forward through the sequence.
- **Transformers (e.g. BERT, GPT):** use self-attention plus explicit positional encodings to represent each word's position.

These models capture **long-range word order** that n-grams miss, at the cost of more parameters and compute.

Demo with notebook

09__text_encoding.ipynb